Course-End Project
Building an RAG Pipeline Using PDF Chunking and Retrieval

Overview

In this project, you will build a Retrieval-Augmented Generation (RAG) pipeline using various tools to process PDF documents, convert them into vectorized chunks, and retrieve relevant information using semantic search. This hands-on project will deepen your understanding of document chunking, embedding, and vector search workflows using real-world content.

Instructions

- **Tools:** Use Visual Studio Code, Python, FAISS, LangChain, and OpenAI API
- **Dataset:** Upload your own set of 2-3 PDF documents (for example, manuals and reports)
- **Submission:** Submit a zipped project folder via the LMS, including your completed notebook, sample outputs, and a short analysis report

Situation

You are working on an AI-powered document assistant tasked with retrieving meaningful insights from long documents. To build this assistant, you must:

- Load and chunk multiple PDF files
- Convert the chunks into embeddings using OpenAI models
- Store the embeddings in an FAISS vector database
- Query the vectorstore to retrieve the most relevant chunk for a given user question

Task

You will build the entire RAG pipeline from scratch in Visual Studio Code, following a step-by-step process:

1. **Chunk and embed PDF content:** Load and chunk your PDF documents, and generate embeddings using OpenAI's embedding model
2. **Store embeddings in FAISS**: Store the document chunks in an FAISS vectorstore to enable similarity-based retrieval
3. **Run retrieval queries:** Accept a sample query and retrieve the most relevant chunk using semantic search

Actions

1. Prepare your PDF data
   - Upload 2-3 PDF files to your working directory in VS Code
   - Organize them inside a dedicated folder

2. Build and run the RAG pipeline in VS Code
   - Set up your virtual environment and install dependencies
   - Load and chunk PDFs using PyPDFLoader and RecursiveCharacterTextSplitter
   - Generate embeddings using Azure OpenAI and store them in an FAISS vectorstore
   - Run retrieval queries with GPT to fetch contextually relevant chunks

3. Analyze the chunking results
   - Print the total number of chunks and the average chunk size
   - Run a sample query and display the top matching result

Result

The final deliverables will include:

1. Completed notebook/scripts consisting of:
   - All code steps shown, executed, and commented on
   - Output cells showing document chunks, results, and queries

2. Uploaded PDFs
   - The 2-3 documents used for testing

3. A project report consisting of:
   - Introduction: Project objective and relevance of RAG
   - Implementation overview: Summary of the pipeline and tools used
   - Metrics: Total chunks, average chunk size, and a sample query output
   - Conclusion: Insights gained, issues faced, and possible improvements